CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

FACULTY OF TECHNOLOGY AND ENGINEERING (FTE)
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Mini-Project Report

MINIBANK

An Object-Oriented Core Banking Console Simulation

Submitted in partial fulfilment of the coursework requirements for

CEUE203: OBJECT-ORIENTED PROGRAMMING

Student Name:	**Vraj Kotadiya**
Enrollment ID:	**D26CE155**
Institute:	CSPIT
Department:	Computer Engineering (CE)
Subject & Code:	Object-Oriented Programming (CEUE203)

Academic Year: 2026–2027

Abstract

MiniBank is an interactive, console-driven core banking simulator written in Java to practically apply and synthesize fundamental and intermediate object-oriented programming concepts within a realistic domain. The application provides comprehensive account management, facilitating the opening of Savings, Current, and Fixed Deposit accounts, processing atomic cash deposits and withdrawals, executing inter-account fund transfers, generating structured financial statements, and compiling summarized account metrics. In addition, the system incorporates custom checked exceptions, runtime field annotations verified via reflection, a generic data repository pattern, contract-based interfaces with default implementations, runtime polymorphism, robust file persistence, and synchronized concurrent transaction execution.

This project report documents the design architecture, core operational workflows, validation logic, concurrency controls, file storage format, and automated verification suites implemented in the system. The demonstration data and console outputs included herein were captured during actual runtime execution with controlled test inputs on the compiled Java source.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Limitations
5. Tools and Requirements
6. System Design
7. Module and Class Description
8. Main Workflows and Algorithms
9. Object-Oriented Concepts Applied
10. Data Persistence and Concurrency
11. Testing and Validation
12. Sample Output Screenshots
13. How to Run the Project
14. Conclusion
15. References

1. Introduction

Financial banking mechanisms serve as an effective model for demonstrating object-oriented system design. Accounts possess internal state, transactional routines alter balances under strict business constraints, and distinct account categories exhibit customized behavioral rules. MiniBank realizes these concepts within a standalone, modular Java application. A command-line user interface captures client interactions and dispatches operations to an underlying service layer, while domain models encapsulate entity behaviors and a generic repository handles in-memory and disk persistence.

The project is engineered specifically for academic demonstration and learning. Account balances are modeled using whole-number units (long values) to prevent floating-point rounding anomalies, and the system operates purely as a local simulation without external banking network dependencies.

2. Problem Statement

Modern application engineering requires developing maintainable, fault-tolerant software that encapsulates sensitive data, validates boundary inputs, enforces specialized business rules across polymorphic entities, and maintains data integrity under simultaneous multi-threaded access. MiniBank addresses this challenge by implementing an integrated banking system that enforces type-specific withdrawal constraints, handles failures through custom exceptions, persists data reliably, and demonstrates advanced Java capabilities including generics, reflection, and concurrency.

3. Objectives

- Model real-world banking entities (Customer, Account, Transactions) using strong encapsulation and inheritance.
- Implement polymorphic account subtypes (Savings, Current, Fixed Deposit) with distinct interest and withdrawal policies.
- Provide core banking functionalities: account creation, deposits, withdrawals, fund transfers, statements, and summary logs.
- Enforce strict input validation using regular expressions and custom runtime annotations (@Positive, @MaxLength).
- Construct a structured checked exception hierarchy with domain-specific properties such as balance shortfall tracking.
- Demonstrate data persistence across application restarts through local file serialization and append-only audit journals.
- Ensure thread safety in concurrent environments using synchronized methods, thread pools, and deadlock-free lock acquisition.
- Validate operational accuracy through comprehensive automated unit tests.

4. Scope and Limitations

4.1 Scope

- Interactive console interface with numbered navigation menus powered by Java switch expressions.
- Domain models representing Savings, Current, and Fixed Deposit accounts with unique ID generation.
- Transactional operations including single-account mutations and atomic cross-account transfers.
- Multi-line statement formatting displaying interest computations, and aggregate statistical reporting.
- File-based persistence mechanism using binary object serialization and NIO DirectoryStream audit logging.
- Multi-threaded batch execution engine with ExecutorService, synchronized balance handling, and deadlock prevention.

4.2 Limitations

- The application is an educational simulation and does not interface with live banking APIs or clearing houses.
- The user interface is terminal-based and does not include a graphical front-end (GUI) or web interface.
- Monetary values are tracked as integer long units rather than fractional decimal currency.
- Authentication is modeled locally for demonstration without multi-factor or biometric security layers.
- The testing suite covers critical operational paths rather than exhaustive enterprise integration scenarios.

5. Tools and Requirements

Component / Environment	Specification & Purpose
Programming Language	Java (Standard Edition)
JDK Version	JDK 17 or higher (Java 26 used in local build environment)
Development Environment	IntelliJ IDEA / VS Code / Command-line Terminal
Build & Compilation	javac compiler / Maven project compatibility (pom.xml)
Automated Testing	JUnit 5 (Jupiter) / Standalone verification test harness
Local Persistence	Binary snapshot (accounts.dat) & text-based logs (activity.log)
Application Model	Modular, multi-threaded console application

6. System Design

6.1 Architecture overview

MiniBank is organized according to layered architectural principles. The presentation layer (MiniBank.java) handles user dialogs and input scanning. The service layer (BankService.java) orchestrates transactional workflows and coordinates repository actions. The data layer (Repository.java) provides generic persistence. Domain entities reside in the model package, error definitions are grouped into the exception package, and utility logic (regex validation, reflective annotation processors, audit loggers) is encapsulated in the util package.

Package / Module	Role & Core Responsibilities
service	Exposes banking APIs, drives transaction workflows, and coordinates multi-threaded workers.
model	Encapsulates account structures, polymorphic variants, customer entities, enums, and records.
repository	Provides generic type-safe CRUD operations, in-memory indexing, and disk persistence.
exception	Contains checked domain exception hierarchy (BankException, InsufficientFundsException, etc.).
util	Supplies regular expression validators, reflective annotation inspectors, and statement formatters.
test	Automated test harness verifying deposit/withdraw limits, concurrency, and persistence.

6.2 High-level workflow

1. System Initialization: Load persisted account entities from disk and instantiate the transaction engine.
2. Header & Menu Rendering: Display bank branding and an interactive numbered operational menu.
3. Input Validation: Read user selection and validate incoming parameters (amounts, account identifiers).
4. Service Delegation: Route commands to appropriate BankService methods enforcing domain rules.
5. Feedback & Exception Handling: Display operation confirmations or handle banking exceptions with shortfall diagnostics.
6. Session Shutdown: Flush memory states to disk storage upon clean user exit.

7. Module and Class Description

7.1 Main menu and service

MiniBank.java represents the application entry point containing the main method. It initializes the BankService, reads console inputs with Scanner, and routes commands using enhanced switch expressions to open accounts, execute deposits/withdrawals, initiate transfers, generate statements, launch batch demonstrations, and exit cleanly. BankService serves as the central business facade coordinating repository access and concurrency locks.

7.2 Account model

Class / Interface	Architectural Purpose in MiniBank
Account	Abstract base class encapsulating accountNumber, ownerName, balance, and active status; implements Transactable & InterestBearing.
SavingsAccount	Subclass enforcing a mandatory minimum balance threshold and offering a 4.0% annual interest rate.
CurrentAccount	Subclass accommodating an overdraft limit up to a designated negative threshold with 0.0% interest.
FixedDepositAccount	Subclass providing 7.0% return while strictly locking withdrawals until maturity.
Transactable	Behavioral interface defining standard deposit and withdraw transactional signatures.
InterestBearing	Interface supplying default methods (yearlyInterest(), monthlyInterest()) to compute financial yields.
WithdrawRule	Functional interface (@FunctionalInterface) facilitating dynamic lambda-based withdrawal validation.
Customer / Address	Represents account holders with personal metadata, static ID generation, and deep-cloning capability.
BankInfo	Immutable Java record containing bank institution branding, branch details, and support information.
MenuOption	Type-safe enumeration representing valid navigational actions across the banking console.
Premium	Marker interface identifying premium customer tiers for specialized operational handling.

7.3 Repository and persistence

Repository<T> establishes a reusable generic container offering save(id, entity), findById(id), and findAll(). In the primary application, the repository dynamically extracts primary keys tagged with @Id via reflection. Accounts are indexed in memory using thread-safe maps and persisted across sessions using StatePersister and append-only transaction logs.

7.4 Exceptions and utilities

BankException serves as the checked base exception class. It is extended by specialized exceptions: InsufficientFundsException (storing exact numerical shortfall), AccountNotFoundException, and InvalidAmountException. The util package contains Validator implementing regex verification and AnnotationValidator which leverages Java Reflection to enforce @Positive and @MaxLength rules.

7.5 Transaction helpers

TransactionProcessor encapsulates a fixed thread pool (4 worker threads) implementing AutoCloseable for graceful resource termination. TransactionBuffer coordinates producer-consumer transaction queuing using synchronized monitors with wait() and notifyAll(). BankSession demonstrates resource management using try-with-resources.

8. Main Workflows and Algorithms

8.1 Opening an account

1. Capture holder name, account category (Savings, Current, Fixed Deposit), and initial deposit amount.
2. Prompt for subtype parameters: minimum balance for Savings, or overdraft threshold for Current accounts.
3. Validate non-empty names, positive initial balances, and minimum deposit criteria.
4. Instantiate the appropriate subclass and persist it in the account repository with an auto-generated identifier.
5. Return confirmation details displaying the created account number.

8.2 Deposit and withdrawal

Deposit operations locate the account by ID, reject non-positive amounts, and atomically increment the balance within a synchronized block. Withdrawal routines evaluate the target account's canWithdraw(amount) implementation: Savings accounts prevent balances from falling below the minimum threshold, Current accounts permit overdrafts up to the authorized limit, and Fixed Deposit accounts reject withdrawals entirely. Any violation throws InsufficientFundsException with calculated shortfall.

8.3 Transfer

Inter-account transfers validate distinct source and destination accounts. To prevent multi-threaded deadlocks during simultaneous bilateral transfers, monitor locks are acquired deterministically in ascending alphanumeric order of account numbers. Funds are withdrawn from the source and deposited into the destination; any failure rolls back the operation cleanly.

8.4 Statement and report

Statement generation formats account details, active status, interest rates, and computed yields using StringBuilder. The reporting routine aggregates total institutional holdings, active accounts, and category distributions across Savings, Current, and Fixed Deposit accounts.

8.5 Batch/concurrency demonstration

The concurrency demo submits 1,000 one-unit deposits to a demo account across multiple worker threads managed by TransactionProcessor. Due to synchronized method execution, the final balance accurately reflects exactly 1,000 units, proving the absence of race conditions. Dynamic lambda rules and reflective audit loggers are also verified.

9. Object-Oriented Concepts Applied

Core OOP Concept	Practical Realization in MiniBank
Encapsulation	Balances and identifiers are declared private; mutations occur solely via verified deposit/withdraw methods.
Abstraction	Account defines an abstract contract with canWithdraw() and interestRate() without revealing subtype logic.
Inheritance	SavingsAccount, CurrentAccount, and FixedDepositAccount inherit core fields from Account via extends.
Polymorphism	BankService handles generalized Account references while Java dispatches subtype methods dynamically at runtime.
Interfaces	Transactable, InterestBearing, and WithdrawRule provide behavior contracts with default method support.
Records & Enums	BankInfo record provides immutable configuration; MenuOption and TransactionType prevent invalid states.
Generics & Collections	Repository<T> handles universal entity persistence; HashMap and ArrayList provide indexing and sorting.
Exception Handling	Checked BankException hierarchy cleanly reports operational

	errors, shortfalls, and missing records.

Annotations & Reflection	Custom annotations (@Id, @Positive, @MaxLength) inspected at runtime via AnnotationValidator.
Multithreading & Concurrency	Synchronized balance mutation, thread-pool task delegation, and lock ordering prevent race conditions.
File Input / Output	ObjectOutputStream serializes accounts to accounts.dat; NIO DirectoryStream audits transaction logs.

10. Data Persistence and Concurrency

10.1 Data file format

MiniBank supports robust persistence across application sessions. In the main application, account records are serialized to a binary data snapshot (accounts.dat) via ObjectOutputStream and reloaded on startup. Additionally, text-based audit journals (activity.log) record all operational events using structured record formats containing timestamp, account ID, operation type, and mutated amount, facilitating subsequent reporting via Java NIO.

10.2 Thread safety

To guarantee integrity under concurrent operations, deposit() and withdraw() methods are synchronized. In-memory accounts utilize ConcurrentHashMap to prevent concurrent modification exceptions. Furthermore, bilateral transfers acquire monitor locks in consistent alphanumeric sequence, completely neutralizing the circular wait condition necessary for deadlocks.

11. Testing and Validation

Verification was carried out using an integrated automated test suite (MiniBankTest.java) alongside scripted console executions. The test harness evaluates deposit increments, overdraft and minimum balance violations, thread-safe synchronization across 10 concurrent worker threads, reflective validation of annotations, generic repository operations, and persistence fidelity.

Automated Test Case	Target Verification Condition	Observed Test Result
depositIncreasesBalance	Depositing 2000 into initial 5000 yields exactly 7000 balance.	PASSED (Balance matches assertion)
withdrawalBeyondLimitThrows	Withdrawing 2500 from 3000 with 1000 minimum balance throws InsufficientFundsException.	PASSED (Shortfall accurately caught)
concurrentDepositsKeepBalance	10 threads each performing 1,000 deposits of 1 unit total exactly 10,000.	PASSED (Zero race condition drift)
persistenceSaveAndReload	Serializing accounts to disk and reloading into fresh array preserves state.	PASSED (Data fully identical)

12. Sample Output Screenshots

The following captures document verified terminal sessions recorded directly from the compiled Java application. The tests demonstrate account creation, financial transactions, formatted statement outputs, and multi-threaded execution.

12.1 Account creation and deposit

```
[Console Output Capture]
Account opened: AC0001 | Vraj Kotadiya | Balance: 5000 | Active: true
Deposit successful! New Balance: Rs. 7000
```

Figure 1. Opening a savings account and executing a cash deposit.

12.2 Transfer and account statement

```
[Console Output Capture]
Transfer successful! Rs. 1000 moved from AC0001 to AC0002.
AC0001 | Vraj Kotadiya | Balance: 6000 | Active: true
Interest rate: 4.00%
Estimated yearly interest: Rs. 240.00
```

Figure 2. Fund transfer confirmation and interest-bearing account statement.

12.3 Account listing and report

```
[Console Output Capture]
AC0001 | Vraj Kotadiya | Balance: 6000 | Active: true
AC0002 | Sample Customer | Balance: 4000 | Active: true
========== BANK REPORT ==========
Total accounts: 2
Total deposits held: Rs. 10000
Savings accounts: 1
Current accounts: 1
Fixed deposit accounts: 0
```

Figure 3. System account inventory and analytical report.

12.4 Concurrency and resource demonstration

```
[Console Output Capture]
Thread-pool demo: 1000 concurrent deposits completed. Final demo balance: 1000
Demo savings account verified thread-safe.
Withdraw rule (anonymous class): true
Withdraw rule (lambda): true
BankSession opened: SESS-101
BankSession closed safely.
[AUDIT] BankService.transfer
[AUDIT] BankService.openSavingsAccount
[AUDIT] BankService.deposit
[AUDIT] BankService.withdraw
```

Figure 4. Multi-threaded batch completion, lambda rules, and audit logging.

13. How to Run the Project

MiniBank requires JDK 17 or newer. In an IDE such as IntelliJ IDEA or VS Code, open the project directory, ensure a JDK 17+ SDK is assigned, and run MiniBank.java directly. In a command-line environment (PowerShell or Bash), the application can be built and launched with standard javac and java commands.

Execution commands for compiling and launching MiniBank via PowerShell:

```
# Compile main application
cd main_application
javac exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java MiniBank.java MiniBankTest.java

# Run interactive console application
java MiniBank

# Run automated verification test suite
java MiniBankTest
```

Additionally, any individual lab practical in practical_wise/ can be compiled and evaluated independently, e.g., javac practical_wise/practical01/*.java followed by java practical01.MiniBank.

14. Conclusion

The MiniBank project synthesizes the core and intermediate competencies of Java object-oriented programming into an interactive, maintainable banking application. Through clean separation of user navigation, service business logic, domain models, and generic repository storage, the project exemplifies real-world modular software architecture. Account subtypes successfully demonstrate polymorphism and inheritance, custom checked exceptions guarantee predictable fault tolerance, and synchronized blocks preserve mathematical precision under heavy multi-threaded workloads. The project fulfills all curriculum objectives for CEUE203, establishing a strong foundation in modern Java software development.

15. References

- CHARUSAT CEUE203 Object-Oriented Programming Laboratory Manual (Odd Semester 2026–27).
- Oracle Java SE Documentation (JDK 17/21): Language Specification & Concurrency Utilities.
- GitHub Project Repository: https://github.com/vrajkotadiya-hub/D26CE155_OOP_MiniProject
- Java Source Packages: main_application/ (model, service, repository, exception, util).
- Automated Test Harness: main_application/MiniBankTest.java and test_all.ps1.